

Migration du POS vers une plateforme SaaS multi-entreprises

Conception du tenant, changement de compagnie, isolation des données, permissions par entreprise, prérequis techniques et plan de mise en œuvre.

Périmètre audité : dépôt POS Laravel situé dans `C:\POS`

État observé : Laravel 10.48.20 · PHP 8.2.24 · application Blade/Vite · MySQL

Mise à jour : 18 août 2026

Statut : architecture et rapport d'avancement — migration fonctionnelle estimée à 74 %, préparation production à 55 %

Sommaire

1. Résumé exécutif	2. Diagnostic du POS actuel
3. Architecture SaaS cible	4. Modèle de données multi-entreprises
5. Connexion et changement de compagnie	6. Rôles et permissions par compagnie
7. Isolation et sécurité	8. Changements par module
9. Préparation aux abonnements	10. Mises à niveau préalables
11. Migration des données	12. Tests et critères d'acceptation
13. Exploitation et déploiement	14. Feuille de route
15. Risques	16. Backlog technique
17. Définition de "prêt SaaS"	18. Références

1. Résumé exécutif

État au 21 août 2026 : le noyau multi-compagnies, le changement de compagnie, l'inscription propriétaire, les rôles/permissions, le storefront et les principaux jobs de notification tenantis sont implémentés. Le flux d'invitation est complet et les écrans Connexion/Inscription possèdent une navigation réciproque. La vue `auth/register` reprend l'interface SaaS complète de `admin/register` et son template POS. Une redirection intelligente conduit désormais chaque utilisateur vers sa première fonctionnalité autorisée après connexion, changement de compagnie ou acceptation d'invitation ; un membre de plusieurs entreprises passe par le sélecteur. La désactivation du cache des pages d'authentification évite aussi l'affichage d'un ancien écran après un retour navigateur. Deux lots de défense par ressource sécurisent désormais Produits, Catégories, Clients et Fournisseurs avec Politiques, autorisations explicites et tests IDOR. Les relations Produit→Catégorie/Fournisseur, Vente→Client et Inventaire→Fournisseur sont contrôlées dans la compagnie active. Les bénéfices sont retirés des écrans, DataTables et PDF lorsque le rôle ne possède pas `reports.view_margin`. La suite automatisée compte **72 tests et 360 assertions réussis**.

Prochaine reprise convenue : commencer par un audit strictement non destructif de l'isolation en base de données : valeurs `company_id` nulles, relations croisées possibles, clés uniques globales, règles de suppression et index manquants. Présenter le rapport avant tout backfill ou migration vers des contraintes `NOT NULL`.

Convention d'attente serveur : les layouts principaux chargent un composant commun qui affiche un loader dans le bouton déclencheur, désactive les doubles clics et restaure l'état en cas d'erreur. Les nouveaux flux AJAX, Fetch ou SweetAlert doivent respecter la convention décrite dans `docs/UI_CONVENTIONS.md`.

1.1 Tableau d'avancement actuel

Chantier	Avancement estimé	État vérifié
Noyau multi-compagnies	92 %	Compagnies, adhésions, contexte, création, sélection, switch et invitations sécurisées opérationnels.
Isolation des données métier	80 %	Scopes et <code>company_id</code> sur les principaux domaines. Politiques et tests IDOR terminés pour Produits, Catégories, Clients et Fournisseurs.
Rôles et permissions	78 %	RBAC par compagnie et interface terminés. La permission dédiée Clients est cohérente entre menu, routes et Policy ; autres Politiques encore nécessaires.
POS, stock et caisse	78 %	Flux principaux testés, relations Vente→Client et Inventaire→Fournisseur sécurisées. Politiques complètes Ventes/Inventaires/Caisses restantes.
E-commerce tenanté	70 %	URL <code>/boutique/{slug}</code> , catalogue, catégories, panier et commande tenantés. Notifications et sécurité avancée à renforcer.
Industrialisation production	25 %	Tests fonctionnels présents ; Redis, stockage privé, MFA, sauvegardes testées, supervision et charge restent à réaliser.
Abonnements	0 %	Reporté volontairement jusqu'à la définition des plans ; créer d'abord <code>EntitlementService</code> .

Estimation globale : migration fonctionnelle SaaS à environ **74 %** ; aptitude à une mise en production SaaS à environ **55 %**.

1.2 Fonctions désormais disponibles

- Inscription créant le compte propriétaire et sa compagnie minimale dans une transaction.
- Navigation claire entre les écrans de connexion et d'inscription, dans les deux sens.
- Compte utilisateur rattachable à plusieurs compagnies avec rôle propre à chaque adhésion.
- Gestion centralisée dans Paramètres : cartes de compagnies, création, confirmation avant bascule et modification de la compagnie active.
- Écran de rôles et permissions avec modules français en accordéons et attribution aux utilisateurs.
- Parcours public d'invitation responsive et contrasté : compte, rôle, expiration, création d'accès et états de sécurité clairement séparés.
- Isolation des principaux modèles métier et protection des routes par contexte et permission.
- Politiques Produits/Catégories, validation anti-référence croisée et tests IDOR sur le catalogue.
- Politiques Clients/Fournisseurs, permission Clients dédiée et contrôles tenant sur les ventes et entrées de stock.
- Boutique publique propre à chaque compagnie avec lien copiable et navigation tenantée par slug.
- Tests automatisés couvrant création/switch, isolation, rôles, ventes, stock, caisse et storefront.

1.3 Diagnostic initial conservé

Le POS actuel est une application essentiellement **mono-entreprise**. La connexion identifie un utilisateur global, mais l'application ne transporte pas systématiquement de contexte d'entreprise. Les produits, catégories, ventes, inventaires, clients, fournisseurs, comptes de caisse et paramètres sont lus sans filtre de compagnie. Plusieurs contrôleurs choisissent même la première entreprise par `CompanySetting::first()`.

Conclusion : ajouter uniquement une liste “Changer d’entreprise” créerait un risque majeur de fuite ou de modification croisée des données. Le passage SaaS exige d’abord un tenant explicite, des clés `company_id`, une portée de requête obligatoire et une autorisation calculée dans le contexte de la compagnie active.

L’architecture recommandée est une **base partagée avec schéma partagé** : chaque ligne métier porte `company_id`. Ce choix convient à l’état actuel du produit, facilite les rapports et réduit le coût opérationnel. La séparation physique par base pourra être envisagée plus tard pour de grands comptes, sans être nécessaire au premier lancement.

L’utilisateur reste global et peut appartenir à plusieurs compagnies grâce à une table d’adhésion. Son rôle, son statut et ses permissions sont rattachés à cette adhésion, pas directement au compte utilisateur. Une session contient l’identifiant de la compagnie active, validé à chaque requête.

Décisions structurantes

Sujet	Décision recommandée	Pourquoi
Tenant	<code>companies</code> + <code>company_user</code>	Relation plusieurs-à-plusieurs utilisateur/compagnie.
Isolation	<code>company_id</code> obligatoire sur toutes les données métier	Filtrage, indexation et contraintes vérifiables.
Contexte actif	Session serveur + middleware <code>ResolveCompany</code>	Compatible avec l’application Blade actuelle.
Autorisation	RBAC par adhésion + Politiques Laravel	Permissions différentes pour un même utilisateur selon la compagnie.
Abonnement	Tables et service d’entitlements préparés, paiement différé	Évite de coupler la migration tenant à un tarif encore inconnu.
Socle	Laravel 13 + PHP 8.4/8.5, après tests de compatibilité	Laravel 10 et PHP 8.2 ne constituent plus un horizon sûr.

2. Diagnostic du POS actuel

2.1 Éléments favorables

- Application Laravel structurée avec modèles Eloquent, contrôleurs, migrations, middleware et jobs.
- Authentification par session déjà présente et Sanctum installé pour de futurs clients API.
- Une entité `company_settings` existe déjà et une relation multi-entreprises partielle existe pour les gestionnaires e-commerce (`ecommerce_managers`).
- Les traitements longs disposent déjà de jobs et la queue est configurée sur la base de données.
- Le rendu PDF existe via `barryvdh/laravel-dompdf` .

2.2 Écarts bloquants

Constat vérifié dans le dépôt	Impact SaaS	Priorité
<code>CompanySetting::first()</code> dans ventes, inventaire, SMS, e-commerce, jobs et tableaux de bord	La compagnie affichée ou notifiée peut être la mauvaise.	P0
Tables métier sans <code>company_id</code> : produits, catégories, ventes, détails, clients, fournisseurs, inventaires, caisses, transactions, promotions, actions...	Aucune frontière de données fiable.	P0
Routes protégées seulement par <code>auth</code>	Un utilisateur authentifié peut atteindre une fonctionnalité non autorisée.	P0
<code>user_type</code> global (1/2/3) et <code>status</code> global	Impossible d'être administrateur dans A et caissier dans B.	P0
Requêtes <code>findOrFail(\$id)</code> sans tenant	Risque IDOR : accès à une ligne d'une autre entreprise via son ID.	P0
Codes uniques globaux (ex. code caisse) et numéros de vente non composés	Collisions entre compagnies et contraintes trop larges.	P1
Fichiers de session et cache locaux	Ne fonctionne pas correctement avec plusieurs instances web.	P1

Très peu de tests métier (tests d'exemple)	Migration risquée, régressions et fuites difficiles à détecter.	P0
Laravel 10.48.20 et PHP 8.2.24	Framework hors support sécurité ; PHP 8.2 en fin de support sécurité le 31/12/2026.	P0

2.3 Dette à corriger pendant la migration

- Ne plus utiliser `created_by` comme substitut de propriétaire d'entreprise : le créateur est un audit, pas une frontière tenant.
- Remplacer les identifiants faibles et champs métier ambigus : `adress`, prix parfois entiers, statuts en chaînes, caisse identifiée par un code global.
- Corriger les migrations non idempotentes ou suspectes (par exemple le `down()` de l'ajout `price_ttc` semble recréer la colonne).
- Retirer les fichiers de diagnostic/miroirs web du document root s'ils ne servent pas en production (`php_error.log`, pages HTML capturées, contenu `public/hub` inutile).

3. Architecture SaaS cible

```

    Navigateur / PWA
      |
    Authentification globale (User)
      |
    ResolveCompany -- vérifie company_user actif
      |
    CompanyContext (company_id immuable pour la requête)
      |
    EnsurePermission -- rôle + permissions de l'adhésion active
      |
    Contrôleurs / Services / Jobs
      |
    Modèles tenantis -- company_id + scopes + Policies
      |
    MySQL partagé | Redis (session/cache/queue) | Stockage objet
  
```

3.1 Composants applicatifs à créer

- `CompanyContext` : objet request-scoped qui expose la compagnie active ; aucune lecture directe de la session dans les services métier.
- `ResolveCompany` : middleware qui charge la compagnie depuis la session, vérifie l'adhésion active et la place dans le contexte.
- `EnsureCompanySelected` : redirige vers le sélecteur si l'utilisateur a plusieurs compagnies ou n'a pas encore de contexte.
- `EnsurePermission:{permission}` et Politiques : contrôle de fonctionnalité et contrôle de ressource.
- `BelongsToCompany` : trait de modèle pour remplir `company_id` à la création et appliquer une portée tenant. Prévoir une API explicite pour les tâches super-admin.
- `SwitchCompanyController` : changement de contexte uniquement vers une adhésion autorisée, régénération de session, purge des caches contextuels et journalisation.
- `EntitlementService` : interface préparatoire qui répond "cette compagnie a-t-elle accès à cette capacité ?". Au départ, elle peut retourner les capacités par défaut.

3.2 Règle de sécurité fondamentale

Toute requête métier doit satisfaire simultanément : utilisateur authentifié + adhésion active à la compagnie courante + permission requise + ressource portant le même `company_id`. Le sélecteur visuel ne constitue jamais une protection.

4. Modèle de données multi-entreprises

4.1 Noyau identité/tenant

Table	Champs essentiels	Contraintes
<code>companies</code> (renommage conseillé de <code>company_settings</code>)	id, uuid/public_id, name, slug, status, timezone, currency, locale, coordonnées, logo_path, timestamps	slug unique ; status indexé ; aucune donnée d'abonnement obligatoire au départ.

company_user	id, company_id, user_id, role_id, status, invited_by, joined_at, last_accessed_at	unique(company_id,user_id) ; FK en cascade contrôlée.
roles	id, company_id nullable, name, key, is_system	unique(company_id,key). Rôles système réutilisables ou clonés à l'onboarding.
permissions	id, key, module, description	key globalement unique et stable dans le code.
permission_role	role_id, permission_id	clé unique composée.
company_invitations	company_id, email, role_id, invited_by, token_hash, expires_at, last_sent_at, accepted_at, declined_at, revoked_at	Jeton seulement sous forme hachée ; cycle de vie complet.

Alternative si des exceptions individuelles sont nécessaires : ajouter **company_user_permission** avec un champ **effect = allow|deny** . Ne l'ajouter qu'après besoin confirmé, car cela complexifie l'explication des droits.

4.2 Tables à tenanter

Domaine	Tables actuelles	Action
Catalogue	categories, products, menu_products, suppliers	Ajouter company_id . Vérifier que produits référencés dans les menus appartiennent à la même compagnie.
Vente	sales, sale_details, clients, code_promos	Ajouter company_id au parent et aux référentiels ; garder aussi sur les détails si utile à l'audit/performance.
Stock	inventories	Ajouter company_id , contraintes et index (company_id, product_id, created_at).
Finance AMS	cash_accounts, transactions, settings	Ajouter company_id et rendre les codes/valeurs par défaut uniques par compagnie.
E-commerce	orders, order_items, ecommerce_managers	orders est déjà lié à la compagnie ; généraliser et remplacer progressivement ecommerce_managers par adhésions/permissions.

Audit et quotas	actions, SMS/WhatsApp counts	Ajouter <code>company_id</code> , actor_id, event, subject type/id, metadata JSON ; séparer les compteurs d'usage des paramètres.
-----------------	------------------------------	---

4.3 Index et contraintes indispensables

- Indexer toutes les colonnes `company_id` et les chemins usuels : `(company_id,status)` , `(company_id,created_at)` .
- Transformer les uniques globaux en uniques composés : `unique(company_id, code)` , `unique(company_id, slug)` , etc.
- Utiliser des clés étrangères composites lorsque possible, ou valider côté service que parent et enfant partagent le tenant.
- Conserver les utilisateurs globaux avec email unique. L'adhésion porte le rôle et l'état dans la compagnie.
- Ajouter un `public_id` UUID/ULID pour les URL publiques ; cela ne remplace pas l'autorisation.

5. Connexion et changement de compagnie

5.1 Parcours attendu

1. L'utilisateur se connecte une seule fois avec son compte global.
2. Le serveur récupère ses adhésions `company_user` actives.
3. Zéro compagnie : écran "aucun accès" / accepter une invitation / créer une compagnie selon la politique commerciale.
4. Une compagnie : sélection automatique et enregistrement de `active_company_id` en session.
5. Plusieurs compagnies : affichage du sélecteur, avec dernière compagnie utilisée proposée.
6. Chaque requête suivante repasse par `ResolveCompany` . Si l'adhésion est révoquée, la session active est supprimée immédiatement.

5.2 Endpoint de switch

`POST /companies/{company}/switch` , protégé par `auth` , CSRF et limitation de débit. Le contrôleur ne doit jamais accepter une compagnie sur la seule base de son ID : il recherche `$user-`

`>companies()->active()->findOrFail($id)` , met à jour la session, régénère son identifiant, journalise l'événement et redirige vers le tableau de bord.

Concurrence multi-onglets : une session partagée signifie qu'un changement dans un onglet affecte les autres. Afficher clairement la compagnie dans l'en-tête, et inclure le tenant attendu dans les formulaires sensibles. Si l'entreprise de la session a changé avant soumission, refuser avec un message demandant de recharger.

5.3 Interface

- Sélecteur permanent dans la barre de navigation : logo, nom, rôle courant.
- Après switch, recharger toutes les données ; ne conserver aucun panier, filtre, cache ou brouillon provenant de l'entreprise précédente sans validation.
- Masquer les menus non autorisés pour l'ergonomie, mais protéger aussi toutes les routes et actions serveur.
- Afficher une bannière distincte pour le super-admin en mode support/impersonation, avec durée limitée et journalisation.

6. Rôles et permissions par compagnie

6.1 Modèle recommandé

Le rôle n'appartient plus à `users.user_type` . Il appartient à `company_user.role_id` . Ainsi, Alice peut être *Propriétaire* dans Compagnie A, *Gestionnaire* dans B et *Caissière* dans C.

Rôle initial	Exemples de droits
Propriétaire	Tous les droits, gestion des membres, sécurité, future facturation ; transfert de propriété protégé.
Administrateur	Configuration opérationnelle, utilisateurs, catalogue, stock, ventes, rapports ; pas forcément facturation.
Gestionnaire	Catalogue, stock, achats, rapports selon besoin.
Caissier	Créer une vente, consulter ses opérations, imprimer un reçu ; remises/annulations limitées.

Comptable / Lecture	Consultation/export finance et ventes, sans mutation.
------------------------	---

6.2 Catalogue de permissions

Adopter des clés explicites et stables, par exemple : `dashboard.view`, `products.view`, `products.create`, `products.update`, `products.delete`, `inventory.adjust`, `sales.create`, `sales.refund`, `sales.export`, `cash.manage`, `reports.view_margin`, `members.manage`, `company.update`, `ecommerce.manage`.

Les Politiques doivent vérifier le tenant de la ressource en plus de la permission. Pour une vente : `membership.company_id === sale.company_id` puis `sales.view`.

6.3 Ordre d'évaluation

1. Compte utilisateur actif.
2. Compagnie active et non suspendue.
3. Adhésion active.
4. Capacité autorisée par le futur plan d'abonnement.
5. Permission fonctionnelle accordée par le rôle.
6. Ressource appartenant à la compagnie active et règles métier satisfaites.

7. Isolation et sécurité

7.1 Défense en profondeur

- **Écriture** : ne jamais accepter `company_id` depuis un formulaire. Le remplir depuis `CompanyContext`.
- **Lecture** : portée tenant obligatoire ; utiliser le route model binding tenanté.
- **Relations** : ne sélectionner les IDs associés que dans la compagnie active (produit, client, fournisseur, caisse, promotion).
- **Jobs** : sérialiser explicitement `company_id`, restaurer le contexte au début du job, vérifier la ressource, puis nettoyer le contexte.
- **Cache** : préfixer toutes les clés par `company:{id}:`. Même règle pour verrous, exports et statistiques.

- **Fichiers** : chemins `companies/{company_uuid}/...`, URL signées et contrôle d'accès avant téléchargement.
- **Logs** : inclure `company_id`, `user_id`, `request_id` ; ne jamais enregistrer mots de passe, jetons ou données de carte.

7.2 Menaces à tester

Scénario	Résultat attendu
Modifier l'ID d'un produit dans l'URL vers celui d'une autre compagnie	404 ou 403, aucune information révélée.
Soumettre une vente avec un produit d'une autre compagnie	Validation refusée et transaction annulée.
Réutiliser un export après révocation de l'adhésion	Lien expiré/refusé ; fichier non public.
Switch pendant qu'un panier est ouvert	Panier vidé ou invalidé ; aucun stock de l'ancien tenant modifié.
Employé sans <code>reports.view_margin</code>	Marge absente des pages, API, exports PDF et notifications.

7.3 Authentification à renforcer

- Utiliser le mécanisme Laravel standard avec throttling ; éviter le flux manuel actuel basé sur `User::where` et `Hash::check`.
- Vérification email, réinitialisation fiable du mot de passe, confirmation du mot de passe pour actions sensibles.
- MFA obligatoire pour propriétaires/super-admins, recommandé pour administrateurs.
- Cookies `Secure`, `HttpOnly`, SameSite approprié ; rotation de session à la connexion et au switch.
- Journaliser connexion, échec, invitation, changement de rôle, switch, export, remboursement et modification de caisse.

8. Changements par module

Module	Changements nécessaires
Dashboard	Statistiques filtrées par compagnie ; caches tenants ; permissions séparées pour CA, profit et marge.
Utilisateurs	Le CRUD gère des membres/adhésions de la compagnie, pas les comptes globaux. Invitation plutôt que création arbitraire d'un mot de passe.
Catalogue / menus	Réalisé pour Produits/Catégories/Fournisseurs : Policies, autorisations explicites, relations limitées à la compagnie active et filtres d'export sécurisés. Étendre ce niveau aux menus.
Inventaire	Transactions atomiques et verrouillage des produits ; historique tenant ; permission distincte pour ajout/retrait.
POS / ventes	Numéro de vente unique par compagnie ; vendeur par <code>user_id</code> en plus du libellé figé ; tous les calculs et caisses dans une transaction DB.
Clients	Réalisé : ClientPolicy, permission <code>clients.manage</code> cohérente dans menu/routes/Policy, IDOR testés et ventes limitées aux clients actifs de la compagnie.
Promotions	Code unique par compagnie ; vérification de dates, limites et tenant.
AMS / caisse	Comptes, réglages et transactions tenants ; écritures immuables ou compensées, journal d'audit.
E-commerce	Réalisé : storefront par slug, commandes et catalogue résolus depuis le tenant public. À poursuivre : notifications tenantes, domaines personnalisés et tests de sécurité complets.
SMS / WhatsApp / e-mail	Quotas et consommation par compagnie ; jobs transportant <code>company_id</code> ; destinataires autorisés dans l'adhésion.
PDF / exports	Logo, devise, données et nom de fichier issus de la compagnie active ; génération asynchrone et stockage privé.
PWA	Clés IndexedDB/cache incluant <code>company_id</code> ; purge au switch ; stratégie hors-ligne tenant-aware.

9. Préparation aux abonnements (tarifs définis plus tard)

Ne pas coder maintenant des prix ni un fournisseur de paiement. Créer une frontière fonctionnelle afin que le tenant et les permissions ne dépendent pas de Stripe ou d'un plan encore inconnu.

Objet préparatoire	Rôle	À faire maintenant ?
<code>plans</code>	code, nom, statut, métadonnées ; aucun prix obligatoire.	Schéma minimal facultatif.
<code>features</code>	Clés de capacité : ecommerce, multi_location, exports, SMS, nombre d'utilisateurs...	Oui, au moins le catalogue de clés.
<code>plan_features</code>	Valeur booléenne ou limite quantitative.	Peut attendre les offres.
<code>subscriptions</code>	company_id, plan_id, status, trial_ends_at, provider refs.	Migration nullable possible ; logique plus tard.
<code>usage_counters</code>	Période, compagnie, métrique, valeur.	Oui pour SMS/WhatsApp et futures limites.
<code>EntitlementService</code>	API unique <code>allows()</code> / <code>limit()</code> .	Oui ; implémentation "capacités par défaut".

Ne pas confondre : le plan dit ce que la compagnie a acheté ; la permission dit ce qu'un membre peut faire. Une fonctionnalité est accessible seulement si les deux l'autorisent.

10. Mises à niveau préalables

10.1 Versions recommandées au 12 août 2026

Élément	Actuel audité	Cible	Approche
---------	---------------	-------	----------

Laravel	10.48.20, support sécurité terminé le 04/02/2025	Laravel 13 si toutes les dépendances le supportent	Faire 10→11→12→13 par branches/commits séparés, en suivant chaque guide d'upgrade. Laravel 12 peut servir de palier, pas de cible longue durée.
PHP	8.2.24 ; sécurité jusqu'au 31/12/2026	PHP 8.4 ou 8.5	Choisir la version supportée par l'hébergeur et les extensions ; tester Dompdf, DataTables et barcode.
Sanctum	3.3.3	Version compatible avec Laravel cible	Utile pour API/PWA ; invalider ou tenanter les abilities si elles portent un contexte.
Frontend	Vite 5, Bootstrap 5.2, Axios 1.6, Sass 1.56	Versions majeures maintenues compatibles	Lancer audit NPM et tests UI ; ne pas mélanger l'upgrade frontend au cœur tenant si évitable.
Dépendances Composer	Contraintes parfois trop larges (<code>yajra/*</code>)	Contraintes majeures explicites	Supprimer <code>*</code> , exécuter audit de sécurité et tests de compatibilité.

10.2 Ordre recommandé

1. Créer une suite de tests de caractérisation sur les flux actuels.
2. Sauvegarder et tester la restauration de la base.
3. Mettre à jour PHP/framework et dépendances sans changement fonctionnel tenant.
4. Corriger dépréciations, migrations et erreurs d'encodage visibles.
5. Stabiliser CI/CD, analyse statique, formatage et audit de dépendances.
6. Commencer ensuite la migration multi-entreprises.

10.3 Contrôles avant toute ouverture Internet

- `APP_ENV=production` , `APP_DEBUG=false` , secrets hors Git, rotation de toute clé déjà exposée.
- HTTPS forcé, en-têtes de sécurité, CORS limité, protection CSRF, limitation de débit.
- Base gérée, chiffrement au repos/en transit, sauvegardes automatiques et restauration testée.
- Redis ou service partagé pour sessions/cache/queue en environnement multi-instance.
- Stockage objet privé pour logos, exports et pièces jointes ; ne pas dépendre du disque éphémère du conteneur.

11. Stratégie de migration des données

Stratégie “expand → backfill → enforce → contract” : ajouter d’abord des colonnes nullable, remplir les données, faire lire/écrire le nouveau modèle, imposer les contraintes, puis retirer l’ancien comportement. Cela réduit l’arrêt de service et permet le retour arrière applicatif.

1. **Inventaire et sauvegarde** : compter les lignes, détecter orphelins/doublons, produire un dump et restaurer dans un environnement de répétition.
2. **Créer le tenant initial** : convertir la ligne actuelle de `company_settings` en première compagnie.
3. **Créer les adhésions** : associer les utilisateurs existants à cette compagnie et mapper `user_type` vers les rôles initiaux.
4. **Ajouter `company_id` nullable** : sur chaque table métier, avec index temporaire.
5. **Backfill** : affecter toutes les données existantes au tenant initial ; propager par relations (`sale_details` depuis `sales`, `order_items` depuis `orders`).
6. **Double protection applicative** : middleware, contexte, scopes, politiques et validation des relations.
7. **Vérification** : zéro ligne sans tenant, zéro relation croisée, totaux financiers et stock identiques avant/après.
8. **Contraintes** : rendre `company_id` non nullable, ajouter FK et uniques composés.
9. **Nettoyage** : supprimer progressivement `CompanySetting::first()`, `user_type` métier et l’ancien pivot e-commerce redondant.

11.1 Retour arrière

Avant l’étape des contraintes, conserver une version applicative compatible et une sauvegarde cohérente. Après mise en production, préférer un correctif en avant. Un rollback de schéma destructif ne doit jamais être automatique. Définir une fenêtre de gel des écritures pour le backfill final si le volume ne permet pas une migration en ligne sûre.

12. Tests et critères d'acceptation

12.1 Matrice minimale automatisée

Niveau	Tests exigés
Unitaires	CompanyContext, calcul des permissions, EntitlementService, génération des numéros par tenant.
Feature	Login à 0/1/N compagnies, switch autorisé/interdit, révocation d'adhésion, middleware sans contexte.
Isolation	Pour chaque ressource : index/show/create/update/delete/export refusent le tenant B depuis A.
Permissions	Chaque route et bouton critique testé avec allow/deny et rôles différents selon deux compagnies.
Transactions	Vente, stock et caisse rollbackent ensemble en cas d'erreur ; tests concurrents sur quantité.
Jobs	Le job restaure le bon tenant ; email/PDF/logo/destinataire appartiennent à cette compagnie.
E2E	Connexion → A → vente → switch B → données distinctes → retour A ; multi-onglets et panier.

12.2 Critères de recette obligatoires

- ✓ Un utilisateur peut avoir des rôles différents dans au moins deux compagnies.
- ✓ Après switch, toutes les pages, recherches, exports, jobs et notifications reflètent la compagnie choisie.
- ✓ Aucun ID d'une autre compagnie ne peut être lu ou modifié, même envoyé manuellement.
- ✓ La révocation d'une adhésion prend effet à la requête suivante.
- ✓ Les totaux vente/stock/caisse de la compagnie historique sont inchangés après migration.
- ✓ Deux compagnies peuvent utiliser le même code métier quand la règle le permet.
- ✓ Les tests d'isolation sont obligatoires dans la CI avant déploiement.

13. Exploitation, performance et déploiement

13.1 Production SaaS

- Instances web stateless derrière un load balancer ; sessions et cache partagés.
- Workers de queue séparés, avec redémarrage contrôlé lors des déploiements et métriques de jobs échoués.
- Migrations compatibles avec déploiement progressif ; pas de renommage/suppression dans le même déploiement que le nouveau code.
- Health checks distincts : application, base, cache, queue et stockage.
- Observabilité : taux d'erreur, latence p95/p99, requêtes lentes, saturation DB, jobs en attente, échecs par compagnie.

13.2 Sauvegarde et continuité

Définir RPO/RTO avant lancement. Recommandation initiale : sauvegardes quotidiennes complètes + journaux/point-in-time recovery si offerts, réplication hors zone, exercice de restauration trimestriel. Prévoir un export logique par compagnie pour portabilité et support, sans promettre une restauration unitaire tant qu'elle n'est pas automatisée et testée.

13.3 Protection des données

Documenter les données personnelles, durées de conservation, suppression/anonymisation, accès support et localisation. Les obligations juridiques dépendent des pays servis : une validation juridique locale sera nécessaire avant commercialisation. Ne pas stocker de données de carte bancaire ; déléguer le paiement futur à un prestataire certifié.

14. Feuille de route recommandée

Phase	Livrables	Sortie attendue
0 — Stabilisation	Tests de caractérisation, sauvegarde/restauration, CI, audit secrets/dépendances, correction migrations.	Base reproductible et mesurable.
1 — Modernisation	PHP 8.4/8.5, Laravel 11→12→13, packages compatibles, tests verts.	Socle maintenu avant modification structurelle.

2 — Noyau tenant	companies, company_user, CompanyContext, middlewares, switch UI, invitations.	Utilisateur multi-compagnies sans encore exposer toutes les fonctions.
3 — Données tenantes	company_id, backfill, scopes, bindings, contraintes et index, suppression des <code>first()</code> .	Isolation complète vérifiée.
4 — Autorisation	Rôles, permissions, Politiques, protection routes/menus/exports/jobs.	Droits propres à chaque compagnie.
5 — Industrialisation	Redis, stockage objet, monitoring, backups, rate limits, MFA, audit.	Exploitation multi-instance sécurisée.
6 — Pilote	Deux compagnies de test, pentest ciblé tenant, charge, runbooks support.	Go/no-go SaaS.
7 — Monétisation ultérieure	Plans, prix, essais, factures, webhook provider, dunning, portail client.	À démarrer lorsque l'offre commerciale est définie.

Position actuelle dans la feuille de route : la phase 2 est largement opérationnelle, les phases 3 et 4 sont avancées mais incomplètes, et la phase 5 doit encore être menée avant un pilote. La modernisation du framework reste à isoler sur une branche dédiée.

Les durées ne sont volontairement pas figées sans connaître l'équipe, le volume de données, les intégrations et le niveau de couverture attendu. Estimer chaque phase après inventaire détaillé et découpage en tickets.

15. Principaux risques et traitements

Risque	Probabilité / impact	Traitement
Fuite inter-tenant par une requête oubliée	Élevé / critique	Scope + Policy + tests systématiques + revue de code dédiée.
Backfill incorrect	Moyen / critique	Répétition sur copie, checksums/totaux, sauvegarde et rapport d'anomalies.

Upgrade framework et tenant simultanés	Élevé / élevé	Phases et commits séparés ; tests à chaque major.
Référence croisée entre tenants	Moyen / critique	Validation des IDs, relations tenantées, contraintes DB quand possibles.
Session active après révocation	Moyen / élevé	Revérifier l'adhésion à chaque requête ; cache d'autorisation très court et invalidable.
Épuisement de ressources par un tenant	Moyen / élevé	Rate limits, quotas, queues, index, limites d'exports et métriques par compagnie.
Échec du switch multi-onglets	Élevé / moyen	Tenant visible, token de contexte sur mutations sensibles, invalidation des brouillons.

16. Backlog technique priorisé

P0 — avant tout pilote SaaS

- ✓ Créer compagnies, adhésions, contexte et switch sécurisé.
- ✓ Mettre en place le RBAC par compagnie et son interface.
- **Partiel** : ajouter et remplir `company_id` sur tout le domaine métier, puis rendre les colonnes obligatoires.
- Éliminer les lectures `first()` et tous les `find*` non tenantés.
- **Partiel** : Politiques et tests IDOR Produits/Catégories/Clients/Fournisseurs terminés ; poursuivre Ventes, Inventaires, Caisses et Commandes.
- Faire transporter `company_id` par tous les jobs, exports et notifications.
- Tester restauration DB et sécuriser sessions, cookies, uploads et exports.
- Mettre à niveau Laravel/PHP sur une branche séparée et sécuriser la chaîne de dépendances.

P1 — avant lancement public

- Redis/session/cache/queue partagés, stockage objet, monitoring et alertes.
- Invitations, vérification email, MFA administrateurs, audit enrichi.
- Transactions/verrous sur ventes, stock et caisse ; idempotence des requêtes critiques.

- ✓ Slug e-commerce par compagnie ; terminer les notifications tenantées et prévoir les domaines personnalisés.
- EntitlementService et compteurs d'usage indépendants du futur tarif.

P2 — après validation commerciale

- Plans, tarifs, essais, coupons, taxes, factures, webhooks et gestion des impayés.
- SSO, multi-sites, analytics avancées, limites personnalisées et éventuelle base dédiée.

17. Définition de “prêt SaaS”

Le projet pourra être considéré prêt à entrer en pilote lorsque toutes les réponses suivantes sont “oui” :

Question	État actuel audité
Chaque donnée métier a-t-elle un tenant explicite et contraint ?	Partiel — principaux modèles tenantés, contraintes finales à terminer
Chaque requête, export et job est-il tenant-aware ?	Partiel — HTTP principal couvert, jobs et exports à auditer
Un utilisateur peut-il avoir un rôle différent par compagnie ?	Oui
Le switch vérifie-t-il l'adhésion et invalide-t-il le contexte précédent ?	Oui
Existe-t-il une suite de tests anti-fuite inter-tenant ?	Oui, couverture encore incomplète — 72 tests / 360 assertions ; Produits, Catégories, Clients, Fournisseurs et bénéfices des ventes couverts
Le framework et PHP sont-ils sur des branches maintenues assez longtemps ?	Non
Les sauvegardes et restaurations sont-elles automatisées et testées ?	À confirmer
Sessions, cache, queue et fichiers supportent-ils plusieurs instances ?	Pas avec les réglages locaux observés

Prochaine action concrète : ne pas reconstruire le noyau, les invitations ni les Politiques Produits/Catégories/Clients/Fournisseurs. Appliquer le prochain lot Policy + autorisations show/history/export + tests IDOR aux Ventes et Inventaires, puis aux Caisses et Commandes. Continuer l'audit des appels à `CompanySetting::first()` avant l'industrialisation Redis, stockage privé, sauvegardes et supervision.

18. Références et périmètre de l'audit

Fichiers du dépôt examinés

`composer.json`, `composer.lock`, `package.json`, routes web/API, modèles, contrôleurs, jobs, services, middleware, migrations, configuration session/cache/queue/base, Docker et Render. Exemples déterminants : `app/Models/User.php`, `app/Models/CompanySetting.php`, `app/Http/Controllers/Auth/LoginController.php`, `routes/web.php` et les migrations métier.

Sources officielles consultées (état au 12 août 2026)

- Laravel — politique de support et versions : <https://laravel.com/docs/12.x/releases>
- Laravel — authentification : <https://laravel.com/docs/12.x/authentication>
- Laravel — autorisation, Gates et Politiques : <https://laravel.com/docs/12.x/authorization>
- Laravel — sessions, cache, queues et base de données : <https://laravel.com/docs/12.x/session> · [/cache](#) · [/queues](#) · [/database](#)
- PHP — versions maintenues et fins de vie : <https://www.php.net/supported-versions.php> · <https://www.php.net/eol.php>

Ce document est une architecture et un plan d'exécution fondés sur une analyse statique du dépôt et des versions locales. Il ne remplace pas une revue de production, un test d'intrusion, une validation juridique/fiscale ni une analyse des données réelles et de leur volumétrie.